

UNIT-5

Object-Oriented Programming (OOP) in Python

It is a programming paradigm that is based on the concept of **objects and classes**, where a program is organized as a collection of objects that contain both data (attributes) and functions (methods).

In OOP, a **class** acts as a blueprint for creating objects, and an **object** is an instance of a class. This approach allows programmers to model real-world entities and their interactions more effectively.

Python supports OOP concepts such as:

- Encapsulation (data hiding)
- Abstraction (hiding implementation details)
- Inheritance (reusing code)
- Polymorphism (multiple forms of methods)

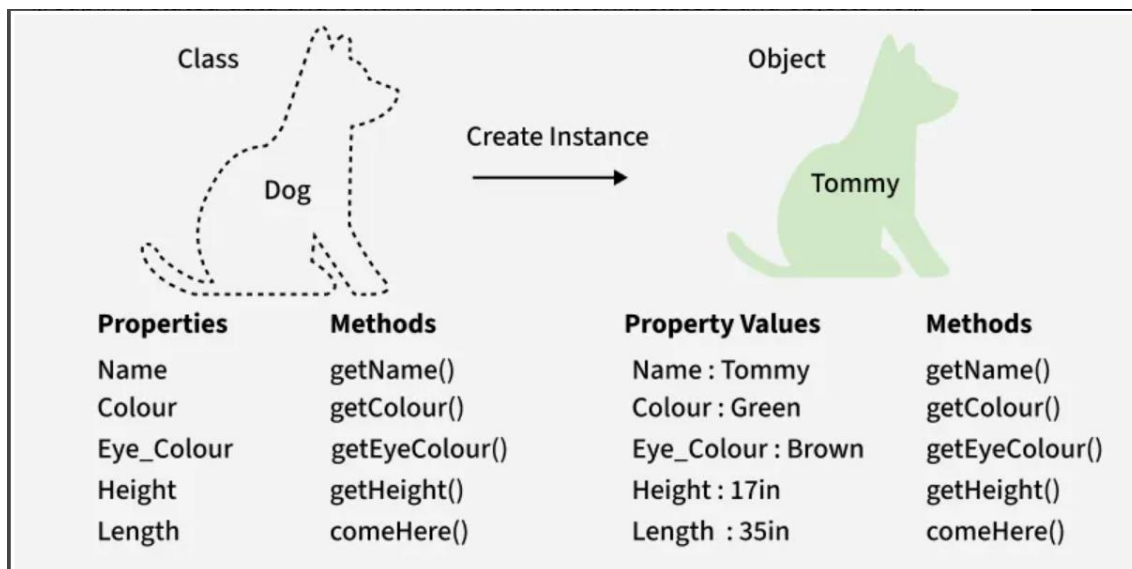
OOP in Python helps in improving code reusability, modularity, and maintainability by structuring programs into reusable and manageable components.

CLASS

A **class** in Python is a user-defined data structure that acts as a blueprint for creating objects. It groups together data (attributes) and functions (methods) that operate on that data into a single unit.

A class in Python is a user-defined template for creating objects. It bundles data and functions together, making it easier to manage and use them. When we create a new class, we define a new type of object. We can then create multiple instances of this object type.

Classes are defined using the `class` keyword followed by the class name and a colon. The body of the class contains variables and methods that describe the properties and behavior of the objects created from the class.



The syntax for defining a class is:

<> Python

```
class ClassName:
    # attributes and methods
```

Attributes are variables that store the state or data of the class, while methods are functions that define the behavior or operations that can be performed on the data.

A class does not occupy memory until objects are created from it. It simply provides a structure or template for creating multiple objects with similar properties and behaviors.

For example:

<> Python

```
class Student:
    name = "John"

    def display(self):
        print("Name:", self.name)
```

In this example, Student is a class that has one attribute (name) and one method (display()).

Thus, defining a class helps in organizing code, improving reusability, and modeling real-world entities effectively using object-oriented programming.

CREATING OBJECTS

An **object** is an instance of a class. It represents a real-world entity and contains both data (attributes) and methods (functions) defined in the class.

An object is a specific instance of a class. It holds its own set of data (instance variables) and can invoke methods defined by its class. Multiple objects can be created from same class, each with its own unique attributes.

The process of creating an object is known as **instantiation**. Objects are created using the class name followed by parentheses.

The syntax for creating an object is:

```
<> Python
object_name = ClassName()
```

When an object is created, memory is allocated for it, and it can access the attributes and methods defined in the class.

For example:

```
<> Python
class Student:
    name = "John"

    def display(self):
        print("Name:", self.name)

s1 = Student()
s1.display()
```

In this example, s1 is an object of the Student class. It can access the class attribute name and call the method display() using the dot (.) operator.

Multiple objects can be created from a single class, and each object can have its own data values. Objects allow interaction with the class structure and help in implementing real-world scenarios in programs.

Thus, creating objects is essential for utilizing the properties and behaviors defined in a class and forms the basis of object-oriented programming.

Data Abstraction and Hiding

Object-Oriented Programming (OOP) in Python is based on important principles such as data abstraction and data hiding, which help in building secure, modular, and easy-to-maintain programs.

Data Abstraction is the process of hiding the internal implementation details and showing only the essential features of an object to the user.

It focuses on *what an object does* rather than *how it does it*.

In Python, abstraction is achieved by:

- Using classes and methods
- Providing a simple interface
- Hiding complex logic inside methods

This allows users to interact with objects without understanding their internal working.

```
</> Python

class Car:
    def start(self):
        print("Car started")

c1 = Car()
c1.start()
```

Here, the user only calls start() without knowing how the car engine works.

Advantages of Abstraction

- Reduces complexity
- Improves readability
- Enhances code reusability

- Provides better security

DATA HIDING

Data Hiding is the mechanism of restricting access to certain data members of a class to prevent unauthorized access or modification.

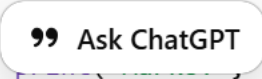
In Python, data hiding is achieved using:

- **Private variables** (prefix with `__`)
- Methods to access private data

This ensures that sensitive data is protected and accessed only in a controlled way.

```
</> Python

class Student:
    def __init__(self):
        self.__marks = 90 # private variable

    def  (self):
        return self.__marks

s1 = Student()
s1.display()
```

Direct access like `s1.__marks` is not allowed.

TYPES OF MEMBERS

- Public Members
- Private Members

DIFFERENCE BETWEEN ABSTRACTION AND DATA HIDING

Feature	Data Abstraction	Data Hiding
Meaning	Hides implementation details	Restricts data access

Feature	Data Abstraction	Data Hiding
Focus	What to show	What to hide
Purpose	Simplify usage	Protect data
Implementation	Methods, interfaces	Private variables

class method and self arguments

In Python, methods inside a class can be categorized based on what they operate on:

- Instance methods (use self)
- Class methods (use cls)

These help in working with object-level data and class-level data respectively.

self Argument (Instance Methods)

- self represents the **current instance (object)** of the class.
- It is used to access **instance variables and instance methods**.
- It must be the **first parameter** in instance methods.
- It is **automatically passed** when an object calls the method.

Example:

```

<> Python

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

s1 = Student("Arun", 85)
s1.display()

```

Explanation:

- `self.name` and `self.marks` belong to a specific object (`s1`).
- Each object has its own data.

Class Method (`cls` Argument)

- A class method works with the **class itself**, not individual objects.
- It uses the `@classmethod` decorator.
- The first parameter is `cls`, which refers to the **class**.
- It is used to access or modify **class variables**.

Example:

```
</> Python

class Student:
    school_name = "ABC School"

    @classmethod
    def change_school(cls, name):
        cls.school_name = name

print(Student.school_name)
Student.change_school("XYZ School")
print(Student.school_name)
```

Explanation:

- `cls.school_name` refers to the class variable.
- Change affects all objects of the class.
- **Differences Between `self` and `cls`**

Feature	<code>self</code>	<code>cls</code>
Refers to	Object (instance)	Class
Used in	Instance methods	Class methods
Parameter name	<code>self</code> (convention)	<code>cls</code> (convention)
Access	Instance variables	Class variables

Feature	self	cls
Decorator needed	No	@classmethod

Initiate Object with `__init__()`

`__init__()` method acts as a constructor in Python and is automatically executed when an object is created. It is used to initialize the attributes of the object with the values provided at the time of object creation.

```
class Dog:
    species = "Canine" # Class attribute

    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute

# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)

print(dog1.name)
print(dog1.species)
```

Output

```
Buddy
Canine
```

Explanation:

- `self` refers to the current object and is used to store data inside it.
- `Dog("Buddy", 3)` creates an object and passes values to `__init__()`.
- `self.name` and `self.age` store the object's name and age.
- `dog1.name` accesses the object's instance attribute and `dog1.species` accesses the shared class attribute.

CLASS AND OBJECT VARIABLES IN PYTHON

In Python, variables defined within a class are categorized into two types:

1. Class Variables – shared by all objects of the class
2. Instance (Object) Variables – unique to each object

These variables help manage data efficiently in Object-Oriented Programming (OOP).

Class Variables

- Class variables are declared **inside the class but outside any method**.
- They are **common to all objects** of the class.
- Only **one copy** of the variable exists, which is shared among all instances.
- They are usually used to store **common properties**.

Example:

```
<> Python

class Employee:
    company = "TCS"    # class variable

e1 = Employee()
e2 = Employee()

print(e1.company)
print(e2.company)
```

Explanation:

- Both e1 and e2 share the same variable company.
- If the class variable is changed, it affects all objects.

</> Python

```
Employee.company = "Infosys"

print(e1.company)  # Infosys
print(e2.company)  # Infosys
```

Instance (Object) Variables

- Instance variables are defined **inside methods using self**.
- Each object has its **own copy** of instance variables.
- They store **object-specific data**.

</> Python

```
class Employee:
    def __init__(self, name, salary):
        self.name = name      # instance variable
        self.salary = salary

e1 = Employee("Arun", 50000)
e2 = Employee("Bala", 60000)

print(e1.name, e1.salary)
print(e2.name, e2.salary)
```

Explanation:

- e1 and e2 have different values.
- Instance variables are independent for each object.
- **Differences Between Class and Instance Variables**

Feature	Class Variable	Instance Variable
Definition	Inside class, outside methods	Inside methods using <code>self</code>
Belongs to	Class	Object
Sharing	Shared by all objects	Unique to each object
Memory	Single copy	Separate copy per object

Feature	Class Variable	Instance Variable
Access	Class name or object	Through object
Example	company	name, salary

the del() method

In Python, del is a **keyword** used to **delete objects, variables, or elements** from memory.

It helps in freeing memory by removing references to objects.

- del deletes a **reference to an object**.
- After deletion, the variable or object **can no longer be accessed**.
- If no references remain, Python automatically **garbage collects** the memory.

Syntax

```
</> Python
del variable_name
```

EX:

```
</> Python
x = 10
del x
print(x)  # Error: x is not defined
```

Explanation:

- Variable x is removed from memory.
- Accessing it later gives an error.

Deleting Object Properties

Python

```
class Student:
    def __init__(self, name):
        self.name = name

s = Student("Arun")
del s.name
```

PUBLIC AND PRIVATE DATA MEMBERS

In Python, data members (variables) of a class can be classified based on their accessibility:

1. Public Data Members – accessible from anywhere
2. Private Data Members – restricted access within the class

This concept is part of encapsulation in Object-Oriented Programming (OOP).

Public Data Members

- Public data members are **accessible from inside and outside the class**.
- Declared normally without any special prefix.
- They can be modified directly.

Example:

Python

```
class Student:
    def __init__(self, name):
        self.name = name  # public variable

s = Student("Arun")
print(s.name)  # Accessible outside class

s.name = "Bala"
print(s.name)
```

Explanation:

- name is public.
- It can be accessed and modified freely.

Private Data Members

- Private data members are **not directly accessible outside the class**.
- Declared using **double underscore (__)** prefix.
- Python uses **name mangling** to protect them.

Example:

```
</> Python

class Student:
    def __init__(self, marks):
        self.__marks = marks    # private variable

s = Student(90)

print(s.__marks)    # Error
```

Explanation:

- Direct access is not allowed.
- Python internally changes `__marks` to `_Student__marks`.

Accessing Private Data Members

- Private variables can still be accessed using **name mangling**:

```
</> Python

print(s._Student__marks)
```

Calling a Class Method from another Class Method

Calling a class method from another class method is a very common pattern, but the “right way” depends on what exactly you mean by *class method* and which language you're using. I'll explain the general idea clearly, then show examples (assuming Python, since that's where this terminology is most common).

A class method:

- Is defined with `@classmethod`
- Takes `cls` as the first parameter (not `self`)
- Belongs to the class, not an instance

So when calling another class method, you usually call it via:

- `cls.method_name()` (preferred inside class methods)
- or `ClassName.method_name()`

Calling One Class Method from Another (Same Class)

<> Python

```
class MyClass:

    @classmethod
    def method_a(cls):
        print("Inside method A")
        cls.method_b()  #  calling another class method

    @classmethod
    def method_b(cls):
        print("Inside method B")
```



Call:

```
</> Python  
MyClass.method_a()
```

Output:

```
Inside method A  
Inside method B
```

Class Method

- A method that works with the **class itself**
- Uses `@classmethod`
- First parameter is `cls`

```
</> Python  
  
@classmethod  
def method(cls):  
    pass
```

Static Method

- A method that is **independent of both class and object**
- Uses `@staticmethod`
- No `self` or `cls`

```
</> Python  
  
@staticmethod  
def method():  
    pass
```

INHERITANCE IN PYTHON

Definition

Inheritance is an Object-Oriented Programming (OOP) concept in which a new class (derived/child class) acquires the properties and behaviors (methods) of an existing class (base/parent class). Theory Explanation Promotes code reusability. Helps in building hierarchical relationships. Allows method overriding. Supports extensibility of existing code. In Python, inheritance is implemented by passing the parent class name inside the child class.

Inheritance is a key concept in object-oriented programming that allows one class (child/derived) to inherit the properties and methods of another class (parent/base). This promotes code reusability and improves maintainability.

Example: Here, a parent class Animal is created that has a method info(). Then a child classes Dog is created that inherit from Animal and add their own behavior.

```
class Animal:
    def __init__(self, name):
        self.name = name

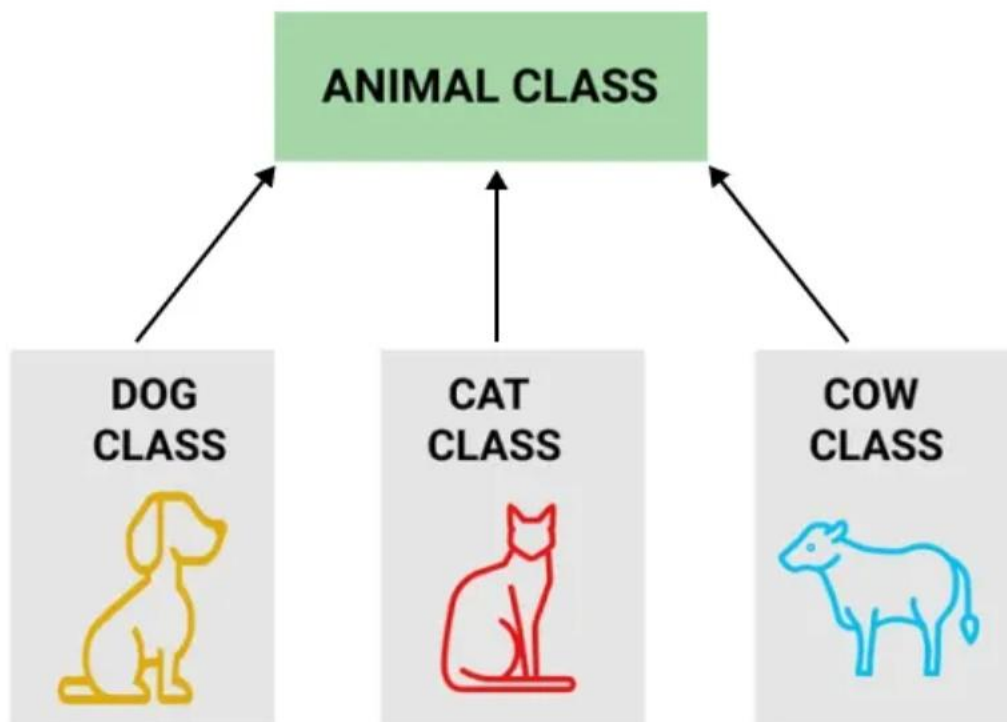
    def info(self):
        print("Animal name:", self.name)

class Dog(Animal):
    def sound(self):
        print(self.name, "barks")

d = Dog("Buddy")
d.info()      # Inherited method
d.sound()
```

Output

```
Animal name: Buddy
Buddy barks
```

Inheritance in Python

Inheritance allows us to define a class that inherits all the methods and properties from another class. **Parent class** is the class being inherited from, also called base class. **Child class** is the class that inherits from another class, also called derived class.

Example Get your own Python Server Create a class named **Person** , with **firstname** and **lastname** properties, and a **printname** method:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)
```

#Use the Person class to create an object, and then execute the printname method:

```
x = Person("John", "Doe")
x.printname()
```

Create a Child Class

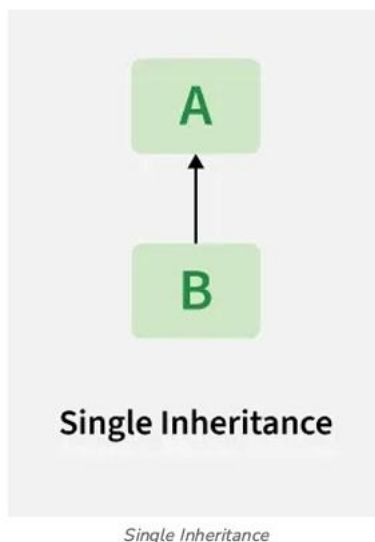
Example

Create a class named **Student**, which will inherit the properties and methods from the **Person** class:

```
class Student(Person):  
    pass
```

Types of Inheritance in Python Types of Inheritance depend upon the number of child and parent classes involved. There are four types of inheritance in Python: Single Inheritance Single inheritance enables a derived class to inherit properties from a single parent class, thus enabling code reusability and the addition of new features to existing code.

1.Single Inheritance



```
# Base class
class Parent:
    def func1(self):
        print("This function is in parent class.")

# Derived class
class Child(Parent):
    def func2(self):
        print("This function is in child class.")

# Driver code
obj = Child()
obj.func1()
obj.func2()
```

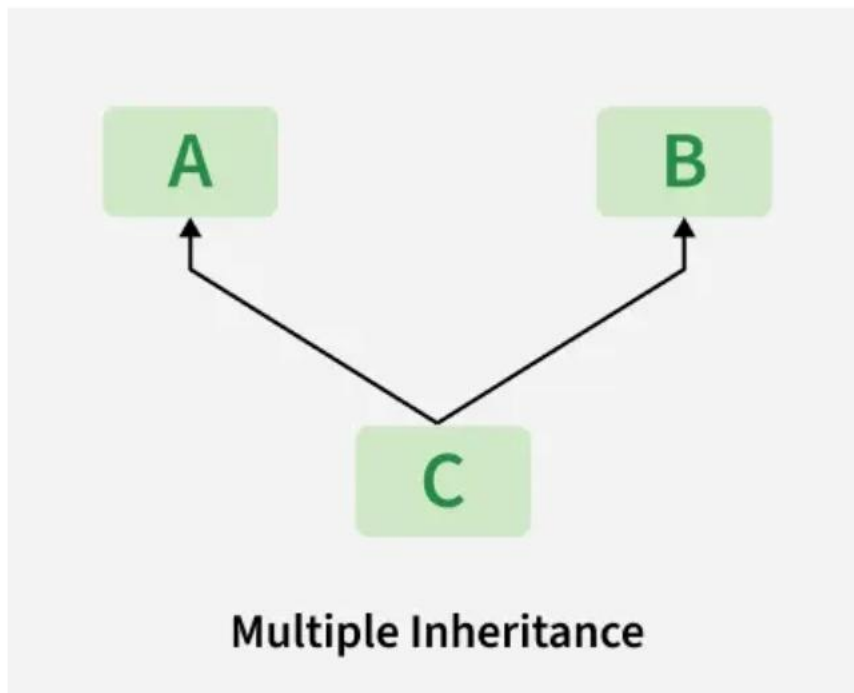
Output

```
This function is in parent class.
This function is in child class.
```

Explanation: The Child class inherits the method func1() from Parent. It also has its own method func2(). This shows how one class can extend another using single inheritance.

2. Multiple Inheritance

When a class can be derived from more than one base class this type of inheritance is called multiple inheritances. In multiple inheritances, all the features of the base classes are inherited into the derived class.



Multiple Inheritance

```
# Base class 1
class Mother:
    mothername = ""

    def mother(self):
        print(self.mothername)

# Base class 2
class Father:
    fathername = ""

    def father(self):
        print(self.fathername)

# Derived class
class Son(Mother, Father):
    def parents(self):
        print("Father :", self.fathername)
        print("Mother :", self.mothername)

# Driver code
s1 = Son()
s1.fathername = "RAM"
s1.mothername = "SITA"
s1.parents()
```

Output

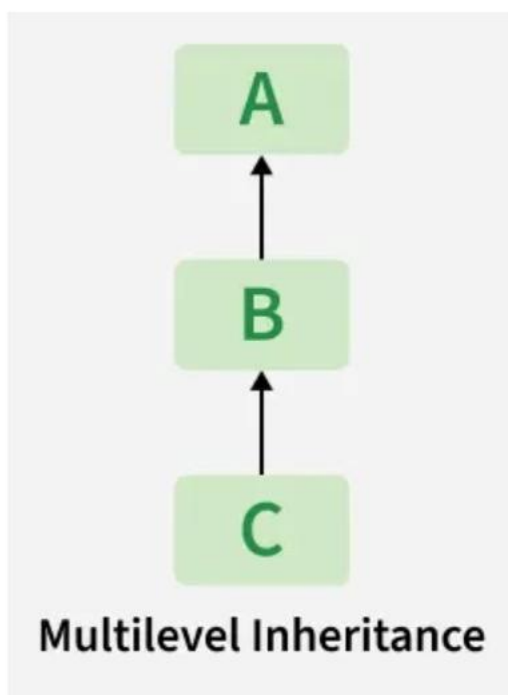
```
Father : RAM  
Mother : SITA
```

Explanation:

- Son inherits from both Mother and Father.
- It can access both mothername and fathername.
- This demonstrates how a class can combine functionalities from multiple sources.

Multilevel Inheritance

In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class. This is similar to a relationship representing a child and a grandfather.



Multilevel Inheritance

```

# Base class
class Grandfather:
    def __init__(self, grandfathername):
        self.grandfathername = grandfathername

# Intermediate class
class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername
        # Call the constructor of Grandfather
        Grandfather.__init__(self, grandfathername)

# Derived class
class Son(Father):
    def __init__(self, sonname, fathername, grandfathername):
        self.sonname = sonname
        # Call the constructor of Father
        Father.__init__(self, fathername, grandfathername)

    def print_name(self):
        print('Grandfather name :', self.grandfathername)
        print('Father name :', self.fathername)
        print('Son name :', self.sonname)

# Driver code
s1 = Son('Prince', 'Rampal', 'Lal mani')
print(s1.grandfathername)
s1.print_name()

```

Output

```

Lal mani
Grandfather name : Lal mani
Father name : Rampal
Son name : Prince

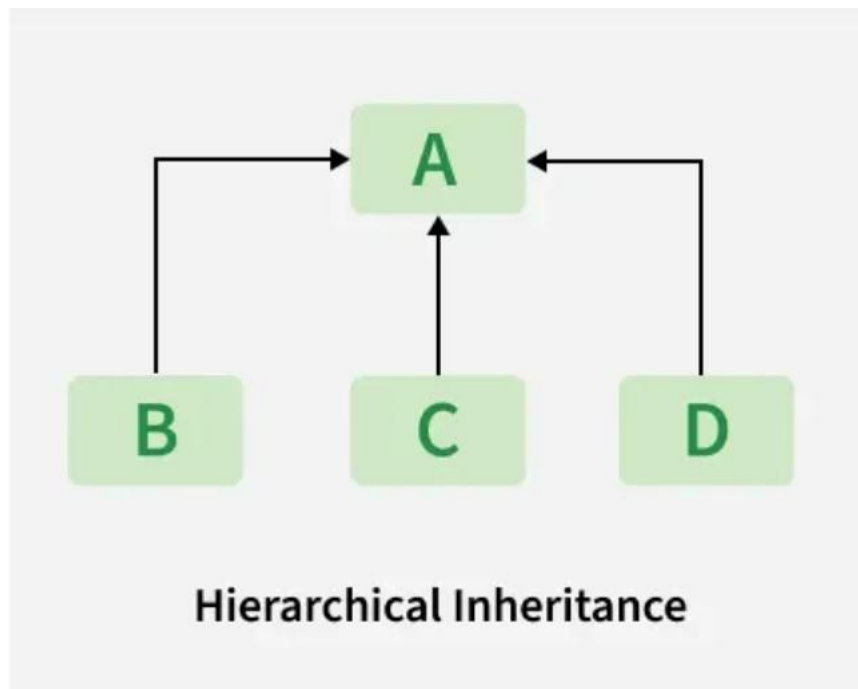
```

Explanation:

- Son inherits from Father, and Father inherits from Grandfather.
- Each constructor passes values up the inheritance chain using explicit constructor calls.
- All ancestor class attributes are accessible from the bottom-most class (Son).

Hierarchical Inheritance

When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance. In this program, we have a parent (base) class and two child (derived) classes.



Hierarchical Inheritance

```
# Base class
class Parent:
    def func1(self):
        print("This function is in parent class.")

# Derived class 1
class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")

# Derived class 2
class Child2(Parent):
    def func3(self):
        print("This function is in child 2.")

# Driver code
object1 = Child1()
object2 = Child2()

object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

Output

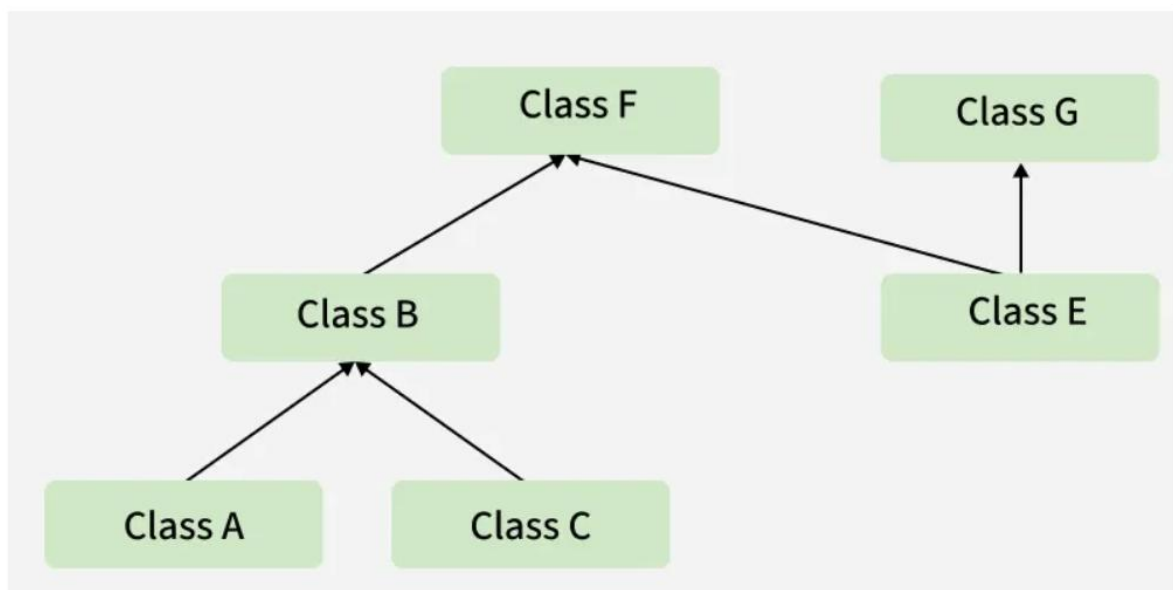
```
This function is in parent class.  
This function is in child 1.  
This function is in parent class.  
This function is in child 2.
```

Explanation:

- Both Child1 and Child2 inherit from the same Parent class.
- Each child can access the func1() method of Parent, but also has its own specific method.
- This pattern is useful when multiple classes need the same base functionality but also have unique behaviors.

Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance. It uses a mix like single, multiple, or multilevel inheritance within the same program. Python's method resolution order (MRO) handles such cases.



Hybrid Inheritance


```

# Base class
class School:
    def func1(self):
        print("This function is in school.")

# Derived class 1 (Single Inheritance)
class Student1(School):
    def func2(self):
        print("This function is in student 1.")

# Derived class 2 (Another Single Inheritance)
class Student2(School):
    def func3(self):
        print("This function is in student 2.")

# Derived class 3 (Multiple Inheritance)
class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")

# Driver code
obj = Student3()
obj.func1()
obj.func2()

```

Output

```

This function is in school.
This function is in student 1.

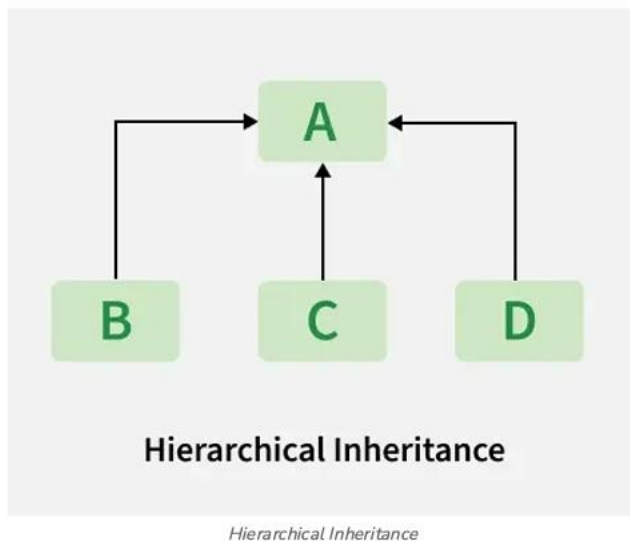
```

Explanation:

- School is the base class inherited by Student1, Student2, and Student3.
- Student1 and Student2 each inherit School (single inheritance).
- Student3 inherits both from Student1 and School (multiple inheritance).
- Python handles method resolution using the MRO, so func1() from School is resolved without ambiguity.

Hierarchical Inheritance

When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance. In this program, we have a parent (base) class and two child (derived) classes



Example:

```
# Base class
class Parent:
    def func1(self):
        print("This function is in parent class.")

# Derived class 1
class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")

# Derived class 2
class Child2(Parent):
    def func3(self):
        print("This function is in child 2.")

# Driver code
object1 = Child1()
object2 = Child2()

object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

Output

```
This function is in parent class.  
This function is in child 1.  
This function is in parent class.  
This function is in child 2.
```

Explanation:

- Both Child1 and Child2 inherit from the same Parent class.
- Each child can access the func1() method of Parent, but also has its own specific method.
- This pattern is useful when multiple classes need the same base functionality but also have unique behaviors.

OPERATOR OVERLOADING:

Operator overloading in Python allows same operator to work in different ways depending on data type. Python built-in data types allow + operator can add numbers, join strings or merge lists and * operator can be used to repeat instances of a string. Python also allows to do operator overloading for user defined classes by writing methods like `__add__()`, `__mul__()`, `__sub__()`, `__lt__()`, `__gt__()` and `__eq__()` to make objects work for operators like +, *, -, and == respectively. We cannot override or change the operator behavior of built-in types (like int, str, list) globally; but we can subclass them (or define your own class) and implement the special methods.

This example shows how built-in operators like + and * work differently depending on the data type.

```
# + operator for integers  
print(1 + 2)  
  
# + operator for strings (concatenation)  
print("Geeks" + "For")  
  
# * operator for numbers  
print(3 * 4)  
  
# * operator for strings (repetition)  
print("Geeks" * 4)
```

Output

```
3  
GeeksFor  
12  
GeeksGeeksGeeksGeeks
```

× ▶ 📄

Operator Overloading for User Defined Types

When we use an operator on user-defined objects, Python doesn't know how to handle it. To make operators work with custom classes, Python provides special methods (also called magic methods).

+ operator -> calls `__add__(self, other)`
- operator -> calls `__sub__(self, other)`
== operator -> calls `__eq__(self, other)`

So, when we write `obj1 + obj2`, internally Python calls:

`obj1.__add__(obj2)`

If this method is defined in the class, operator overloading works.

Example 1: This code shows how to overload the `+` operator for integers and strings inside a custom class.

```
class A:
    def __init__(self, a):
        self.a = a

    # define behavior of +
    def __add__(self, o):
        return self.a + o.a

ob1 = A(1)
ob2 = A(2)
ob3 = A("Geeks")
ob4 = A("For")

print(ob1 + ob2)    # integer addition
print(ob3 + ob4)    # string concatenation

# actual working (internally)
print(A.__add__(ob1, ob2))
print(ob1.__add__(ob2))
```

Output

```
3
GeeksFor
3
3
```

Explanation:

- `ob1 + ob2` automatically calls `ob1.__add__(ob2)`.
- Python translates it into `A.__add__(ob1, ob2)`.
- first operand (`ob1`) becomes `self` and second operand (`ob2`) becomes `other`.

× ▶ 📄

Overloading Comparison Operators

Operators like >, <, and == can also be overloaded.

Example 1: This code shows how to overload > operator to compare two objects based on their stored values.

```
class A:
    def __init__(self, a):
        self.a = a

    def __gt__(self, other):
        return self.a > other.a

ob1 = A(2)
ob2 = A(3)
if ob1 > ob2:
    print("ob1 is greater than ob2")
else:
    print("ob2 is greater than ob1")
```

Output

```
ob2 is greater than ob1
```

REVERSE ADDING (____radd__)

Definition

__radd__() is called when the **left operand does not support addition**, and Python tries the right operand.

- Used when **object is on right side of operator**.
- Useful when combining **different data types**.

Example Program

```
class Num:
    def __init__(self, value):
        self.value = value

    def __radd__(self, other):
        return self.value + other
```

4. OVERRIDING `__getitem__()` AND `__setitem__()`

`__getitem__()`

- Used to access elements using indexing.
- Called when we use `obj[index]` .

`__setitem__()`

- Used to assign values.
- Called when we do `obj[index] = value` .

Example Program

```
class MyList:
    def __init__(self, data):
        self.data = data

    def __getitem__(self, index):
        return self.data[index]

    def __setitem__(self, index, value):
        self.data[index] = value

obj = MyList([1, 2, 3])
print(obj[1]) # calls _getitem_ obj[1]
obj[1] = 10 # calls _setitem_ print(obj.data)
```

5. OVERRIDING in OPERATOR (____contains____)

Definition

____contains____() is used to define behavior of the in operator.

- Checks membership in an object.
- Returns True or False .

Example Program

```
class MyCollection:
    def __init__(self, items):
        self.items = items

    def __contains__(self, item):
        return item in self.items
```

6. OVERRIDING `__call__()` METHOD

Definition

`__call__()` allows an object to be **called like a function**.

- Makes objects behave like functions.
- Useful in **function objects, decorators, callbacks**.

Example Program

```
class Multiply:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, value):
        return value * self.factor
```